
ICPC Algorithm Library

目录

1	Basic	1
1.1	test	1
1.2	pai	1
1.3	run	1
1.4	缺省源	1
2	Geometry	1
2.1	vector	1
2.2	seg_poly	1

1 Basic

1.1 test

```
export fs="-fsanitize=address,undefined"
ulimit -s 1024000
ulimit -v 1024000
mk() { g++ -o $1 $1.cpp -O2; }
```

1.2 pai

```
pai() {
    mk a; mk bf; mk gen
    while true; do
        ./gen > 1.in
        ./bf < 1.in > 1.ans
        ./a < 1.in > 1.out
        if diff -Zq 1.out 1.ans; then echo ac; else echo wa; break; fi
    done
}
```

1.3 run

```
shopt -s nullglob # 没有匹配到任何文件时，让它返回空
ulimit -s 1024000
g++ $1.cpp -o $1 -std=c++20 -O2 -fsanitize=address,undefined || exit
for f in "$@"/*.*; do
    x=${f%.in}
    \time -f "$x: %es %MKB" ./$1 < $x.in > $1.out
    diff -Zq $1.out $x.ans
done
```

1.4 缺省源

```
#include <bits/stdc++.h>
using namespace std;
#define ALL(s) s.begin(), s.end()
#define SZ(s) int(s.size())
#define pb push_back
using LL = long long;
using ULL = unsigned long long;
using I = __int128;
void Cas() {
}
int main() {
    cin.tie(0)->sync_with_stdio(0);
    int t = 1;
    cin >> t;
```

```
while (t--) {
    Cas();
}
return 0;
}
```

2 Geometry

2.1 vector

```
using db = long double;
struct p2 {
    db x, y;
    db len() { return hypot(x, y); }
    p2 operator+(p2 b) { return {x + b.x, y + b.y}; }
    p2 operator-(p2 b) { return {x - b.x, y - b.y}; }
    p2 operator*(db k) { return {x * k, y * k}; }
    p2 operator/(db k) { return {x / k, y / k}; }
    db operator*(p2 b) { return x * b.x + y * b.y; }
    db operator%(p2 b) { return x * b.y - y * b.x; }
};
using ps = vector<p2>;
```

2.2 seg_poly

```
// pa 到 pb 的叉积，逆时针转为正
db side(p2 p, p2 a, p2 b) { return (a - p) % (b - p); }
db tol(p2 p, p2 a, p2 b) { return abs(side(p, a, b)) / (b - a).len(); }
```